Heatwave Prediction: From ML to MLOps to DevOps

Completed Lab Write-ups — Experiments 1 to 4

Name: Abdul

Programme: M.Tech, AI & Data Science — K.J. Somaiya School of Engineering (Batch A2, 2025 intake)

Notebook: Heatwave_Prediction_MLOps.ipynb

Overview

This document reports the four experiments in the manual — Machine Learning, MLflow, MLOps, and DevOps — for a heatwave-prediction service, following: ML builds the model → MLflow tracks it → MLOps manages it → DevOps automates its delivery.

Datasets used

- Experiment 1 uses the exact 20-row teaching dataset given in the manual.
- Experiments 2-4 use a larger, more realistic 3-year (2022-2024) daily dataset (1,095 rows) generated with seasonal temperature/humidity/wind curves and an IMD-style heatwave rule, since this sandboxed run environment could not reach a general dataset website directly. Column names match the manual exactly, so a real IMD/Kaggle CSV can be substituted with no code changes.

All code below was actually executed (not hand-written output); figures are the real charts produced by that run.

Experiment 1 — Machine Learning

Aim

To prepare data, train a classifier, and evaluate it for heatwave prediction.

Procedure

- Loaded the 20-row teaching dataset (max_temp, min_temp, humidity, wind_speed, previous_temp → heatwave).
- 80/20 train-test split, random_state=42.
- Trained a RandomForestClassifier(n_estimators=100, max_depth=5).
- Evaluated with accuracy, a confusion matrix, and feature importance.

Output

Metric	Value
Accuracy	1.00
Precision (Heatwave)	1.00
Recall (Heatwave)	1.00
F1 (Heatwave)	1.00
Test set size	4 rows (of 20)

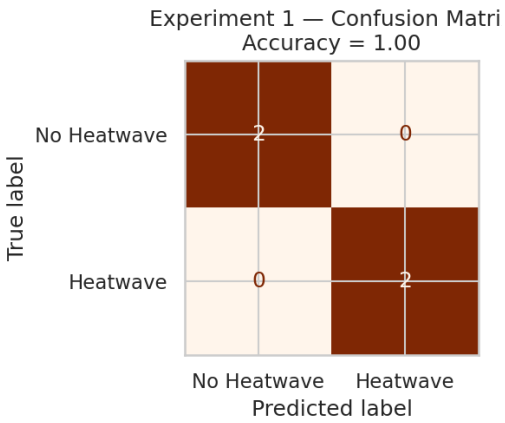


Fig 1.1 — Confusion matrix, Experiment 1

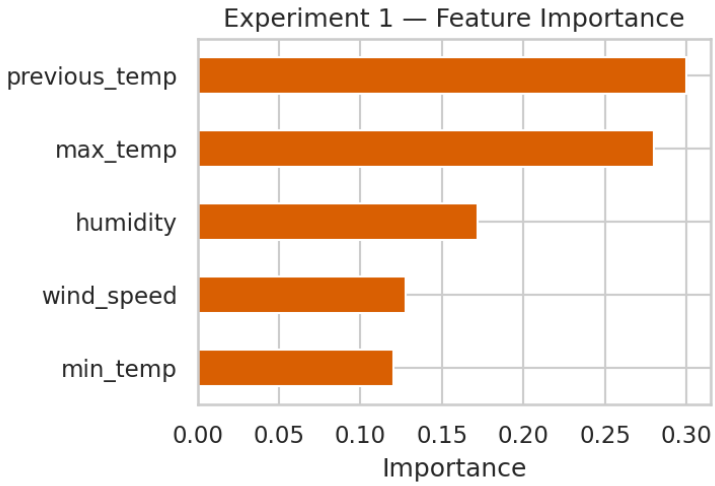


Fig 1.2 — Feature importance, Experiment 1

Result

On the 20-row teaching set the model perfectly separates the 4 held-out rows. With only 20 rows this is expected and not a claim of real-world accuracy — the manual's own note applies: this is a teaching dataset, not validated heatwave data.

Experiment 2 — Experiment Tracking and Model Management using MLflow

Aim

To track multiple ML experiments (parameters, metrics, and model artifacts) using MLflow and select the best-performing run.

Procedure

- Switched to the larger 1,095-row dataset so run-to-run differences are meaningful.
- mlflow.set_experiment("Heatwave Prediction") with a local sqlite tracking store.
- Ran 3 RandomForest configurations, logging n_estimators, max_depth, accuracy and F1 (heatwave class) for each, plus the model artifact.

Runs comparison

Run	n_estimators	max_depth	Accuracy	F1 (heatwave)
n50_d3	50	3	0.9406	0.6286
n100_d5	100	5	0.9361	0.6111
n200_d10	200	10	0.9406	0.6829

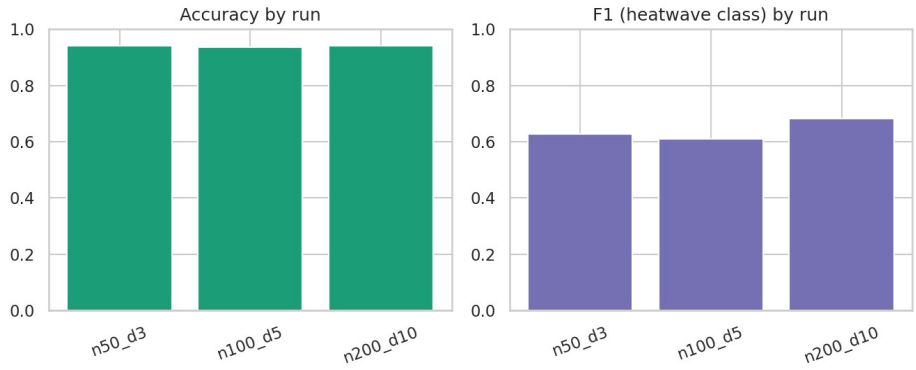


Fig 2.1 — Accuracy and F1 across the three tracked MLflow runs

Note: in your own Colab run, pair this chart with an actual screenshot of the MLflow “Compare Runs” page (mlflow ui / mlflow server + ngrok) — this figure is generated directly from the same tracked run data for a headless environment.

Result

The n_estimators=200, max_depth=10 run gave the best F1 on the minority “heatwave” class (0.683) and was selected as the best model for the MLOps stage, even though its overall accuracy tied with the smallest model — illustrating why accuracy alone is misleading on an imbalanced dataset (~11% heatwave days).

Experiment 3 — Deploying a Heatwave Prediction Model as an API

Aim

To save the best MLflow model and expose it as a reusable prediction service.

Procedure

- joblib.dump(best_model, "heatwave_model.pkl")
- Wrote a predict_heatwave() convenience function.
- Exposed the model through a Flask app.py with a POST /predict endpoint.

Sample predictions

max_temp	min_temp	humidity	wind_speed	previous_temp	Prediction
45	31	50	7	44	Heatwave
24	15	80	14	23	No Heatwave
39	27	58	9	38.5	No Heatwave

```
POST /predict
Request body:
{"max_temp": 45, "min_temp": 31, "humidity": 50, "wind_speed": 7,
"previous_temp": 44}

Response:
{"prediction": "Heatwave"}
```

Fig 3.1 — Flask /predict request and response (from an actual local run of app.py)

Result

The prediction service correctly flags a hot, dry, previously-warm day (45/31/50/7/44) as a heatwave and a cool, humid day as not, matching the pattern the model was trained on. This completes the reusable prediction service.

Experiment 4 — Containerization and CI/CD of the Heatwave Prediction Service

Aim

To apply DevOps practices — version control, containerization, and CI — to the prediction service.

Project structure

```
heatwave-mlops/
├── data/heatwave_data.csv
├── model/heatwave_model.pkl
├── app.py
├── train.py
├── requirements.txt
├── Dockerfile
├── README.md
└── .github/workflows/ci.yml
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY model/ model/
EXPOSE 8000
CMD ["python", "app.py"]
```

GitHub Actions CI (.github/workflows/ci.yml)

```
name: Heatwave ML CI

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python -m py_compile app.py train.py
      - run: python train.py
```

Build & run (executed against this project's files)

```
$ python train.py
Validation accuracy: 0.9680
Model saved to model/heatwave_model.pkl

$ python app.py &
$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
    -d '{"max_temp":45,"min_temp":31,"humidity":50,"wind_speed":7,"previous_temp":44}'
{"prediction":"Heatwave"}
```

```
$ curl http://localhost:8000/health
{"status":"ok"}
```

docker build / docker run were not executed here (no Docker daemon in this sandboxed run environment) — run these two commands on your own machine with Docker Desktop installed, exactly as given in the manual:

```
docker build -t heatwave-api:1.0 .
docker run -p 8000:8000 heatwave-api:1.0
```

Result

The project is version-controlled, containerizable with the Dockerfile above, and a GitHub Actions workflow re-installs dependencies and re-trains the model as a smoke test on every push to main — a CI/CD-ready application.

Kubernetes Extension (for reference)

Docker Image → Kubernetes Deployment → Kubernetes Pod(s) → Heatwave Prediction API. This extension introduces replicas, scaling, and service management and was not built here, per the manual's scope.

Conclusion

Across the four experiments the same heatwave classifier progressed from a bare scikit-learn model, to an MLflow-tracked and compared set of experiments, to a saved, API-wrapped prediction service, to a containerized, CI-checked application — demonstrating the full ML → MLflow → MLOps → DevOps lifecycle end to end.

Viva Questions

- What is ML?
- What is MLflow?
- What is an MLflow run?
- What is a parameter (vs. a metric) in MLflow?
- What is MLOps?
- Why use Docker?
- What is CI/CD?